

Siófoki SZC Mathiasz János Technikum és Gimnázium

Szakképzés neve: Szoftverfejlesztő

OKJ száma: 54 213 05

ZÁRÓDOLGOZAT

NCBS

Készítette:

Csengeri Miklós László

Konzulens:

Péterffyné Takács Katalin

Balatonboglár

2024.

ZÁRÓDOLGOZAT – FELADAT KIÍRÁS

Hallgató: Csengeri Miklós László

Záródolgozat címe: NCBS (Nem Csak Ballisztikus Szimulátor)

Szakképesítés megnevezése: Szoftverfejlesztő

A szakképesítés azonosító száma: 54 213 05

Konzulens: Péterffyné Takács Katalin

A téma részletezése: Godot Játékmotorhoz C# és GDScript nyelven írt lövedék szimuláló szoftver játékfejlesztőknek.

Beadási határidő: 2024. április 13.

.....
konzulens

A témakiírást átvettem:

.....
hallgató

Balatonboglár, 2023. december 13.

Tartalom

NCBS	3
Kiemelt Funkciók	4
Pontos 2D ballisztika.....	4
Nagy pálya.....	4
Indoklás	4
Fő erőforrások	5
A Godot Játékmotor	5
Godot Játékmotorhoz letöltött erőforrások	11
Visual Studio Code.....	15
GDScript.....	15
C#	15
Blender	15
Audacity	16
GitHub	16
Obsidian	16
A Godot problémái	17
Felhasználói Dokumentáció	18
Rendszerkövetelmények.....	18
Minimum Gépigény	18
Ajánlott Gépigény	18
Menü.....	19
Mozgás és kamera	20
Lövés és célzás	20
Fejlesztői dokumentáció.....	20
TimeManager	21
SettingsManager.....	21
DebugTools	21
AudioMixer	22
NCBS	22
Menu:	22

BasicVersionNamer:	22
Sway:	22
tank_cannon_angle:	22
pause_menu.gd:	22
slider_selector.gd:	22
static_target.gd	23
target.gd	23
target_destructible.gd	23
camera_bullet.gd:	23
game_camera.gd	23
spectator_camera.gd	24
fps_gun_base.gd	24
audio_adder.gd	24
NCBS.cs:	25
NCBS.gd	25
NCBS_world.gd:	25
<i>NCBSWorldRes.cs</i>	25
NCBSBullet.cs:	25
<i>NCBSHitRes.cs</i>	26
Asset források és licenszek	30
Források	30
Irodalom Jegyzék	31
Utószó	31
Ábrajegyzék	32

NCBS

A játékban képesek vagyunk számos különböző reaktív és statikus célpontra lőni jelenleg egy távcsővel rendelkező forgó-toló záras puskával, amit valódi referenciákkal Blenderben modelleztem és animáltam. A játék a Godot játékmotor segítségével lett létrehozva ezek mellett számos más szoftver is állt rendelkezésemre. Ezekről mind pár szót ejteni fogok.

Viszont először arról is szeretnék beszélni mi a játék célja és ötlete.

Kiemelt Funkciók

Pontos 2D ballisztika

A játék ballisztikáját egy 2D modelről mintáztam amit kézi fegyverekhez gyakran alkalmaznak. Akár puskához csinált okos távcsövekben is. A ballisztikai modellek közül ennek van a legjobb teljesítménye. Viszont ez a legkevésbé pontos is. Nagy Szögeken és nagy távokon a pontossága csökken. De kisebb távokon ez nem probléma (Kevesebb mint kb. 1.5 kilométer). A legnagyobb limitációja a modelnek hogy csak 2 dimenziós. 3 dimenziós modellé is át tudnám alakítani de a pontosság úgy valószínűleg romlana mivel ha valódi modelről mintáznám akkor nagyon sok dolgot át kéne írnom. Attól még meg szeretném csinálni. Erre a model félig fel van készítve például a golyónak vannak olyan adatmezői amik nincsenek használva a 2Ds modell miatt.

Ezt azért tettem bele mivel majdnem minden videójátékban még azokban is amik realisztikusak szeretnének lenni nem gyakori realisztikus ballisztikát találni. Általában nagyon le van szimplifikálva. Ez a fő probléma.

Nagy pálya

A pálya 6 kilométer átmérőjű ha valaki szeret sétálni akkor jó sokáig tud.

Ez a méret azért lett választva mivel ez közel a maximum táv amit a puska szimulált lövedéki meg tud tenni. Így ha nem direkt szeretnénk a pályából kilőni nem nagyon fog meggesni.

Indoklás

A játékot azért hoztam létre mivel szeretem a realisztikus játékokat és a játékfejlesztést.

Korábban már csináltam kisebb játékokat Unityvel de 2023 szeptemberében egy változtatást hirdettek a felhasználási feltételekhez aminek a fő változása hogy bizonyos számú letöltés után minden utána következő letöltésért fizetnünk kell. Azóta ez a téma már frissült de a Unityben lévő bizalmat sokan elvesztették. Emiatt más játékmotorok felé néztem ezekből a Godoton kívül az Unreal Engineet emelném ki. A Godot és az Unreal között nagy a különbség míg a Godot 2Ds teljesítménye nagyon jó és kis mérete és erőforrás igénye is előny, Unreal Engine ehhez képest gyönyörű magas minőségű 3Ds játékok fejlesztésére alkalmas. Ezen felül míg Unreal Engineben C++-ban és a saját vizuális programozási nyelvével lehet programozni Godotban akármilyen programozási nyelvet lehet használni(amíg van hozzá GDExtension írva). Viszont a játékmotorban C++-hoz(A játékmotor forráskódjának nyelve), GDScripthez(A játékmotor saját nyelve) és végül a C#-hoz van támogatás alaptól.

Végül a Godotot választottam. Szerintem egy játéknak nem kell realisztikusan kinéznie hogy jó játék legyen sőt a kedvenc játékaimból kevésben van realisztikus grafika. Emellett mivel Godot teljesen nyílt forráskódú akármennyit keresel akármit csinálsz a

motorral teljesen legális és egy fillért sem kell fizetni érte. Idővel a Godotból olyan játékmotor lesz ami képes harcolni a nagy motorokkal és ha már most elkezdem akkor akár későbbiekben egy játékfejlesztő cégnél is dolgozhatok tapasztalattal. Így ez a project tanuló project is volt nekem mivel ezelőtt csak kis dolgokat írtam Godotban. A programozási nyelvnek a GDScriptet és a C#-ot választottam mivel a GDScript nagyon szorosan bele van integrálva a motorban és gyors vele fejleszteni a C#-al meg nagy teljesítményű kódot lehet írni és már tanultam nem úgy mint a C++-t vagy akár a Rustot. Így utólag szerintem jobban választottam mint hogyha ugyanezt a projectet Unityben csináltam volna ahogy azt az elején terveztem.

De csak Godot-tal nem tudtam volna megcsinálni a projectet ezért sok másik erőforrást is használtam.

Fő erőforrások.

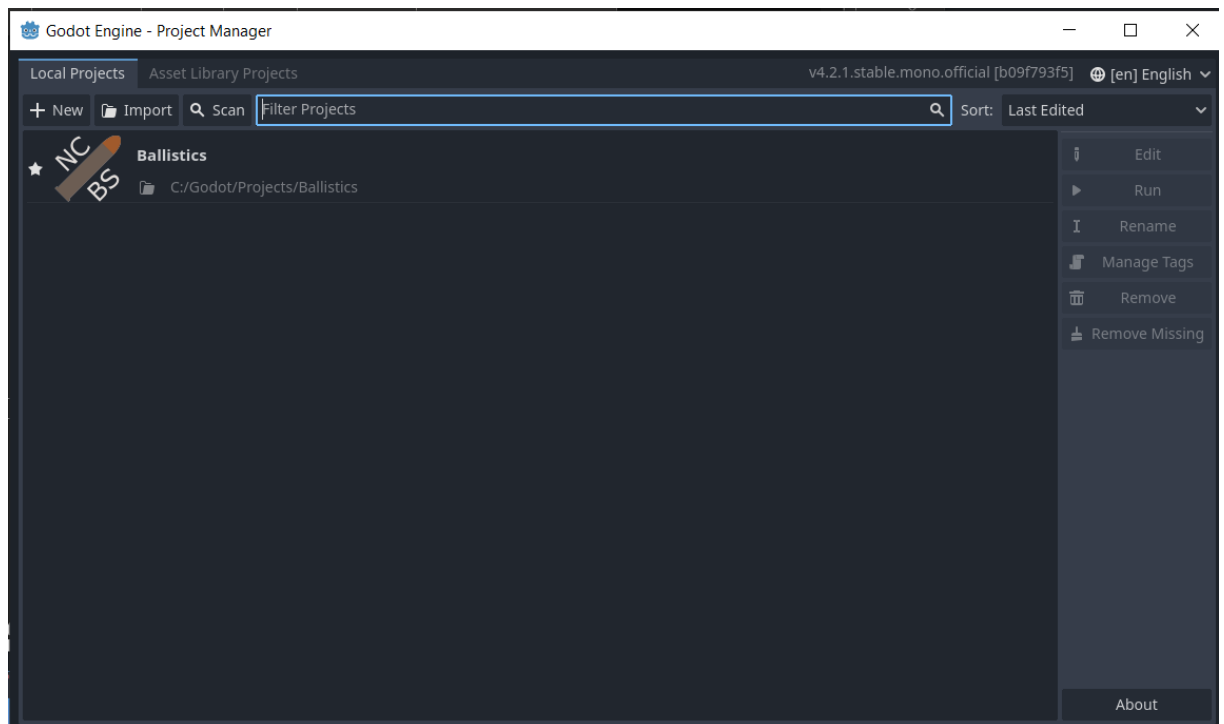
- Godot Játékmotor
- Godot Játékmotorhoz letöltött erőforrások
- Visual Studio Code
- GDScript
- C#
- Blender
- Audacity
- Github
- Obsidian
- Ezek mellet sok speciális vagy keveset használt erőforrást is használtam.

A játékhoz majdnem minden erőforrás nyílt forráskódú kivéve a GitHubot és az Obsidiant. Viszont mindkettőt lecserélhettem volna egy nyílt forráskódú alternatívára. pl. GitLab és Logseq.

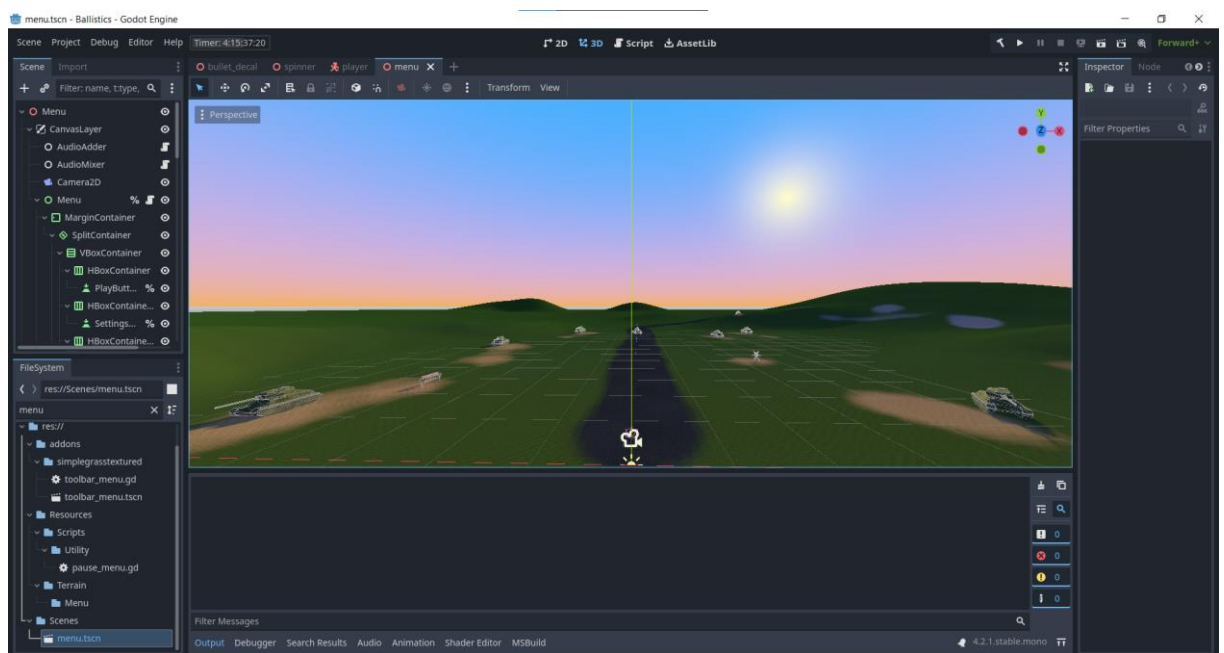
A Godot Játékmotor

A Godot egy teljesen nyílt forráskódú és ingyenes játékmotor amiben mind 2D és 3D játékokat is tudunk fejleszteni emellett sima szoftverek írására is teljesen alkalmas. Minden gyakori platformra lehet vele programot írni emellett egyre több stúdió használja és a népszerűsége folyamatosan nő. Elsődlegesen számítógépre lett tervezve de a Godot editor akár Androidra is elérhető így telefonról lehet írni akár számítógépre is játékot.

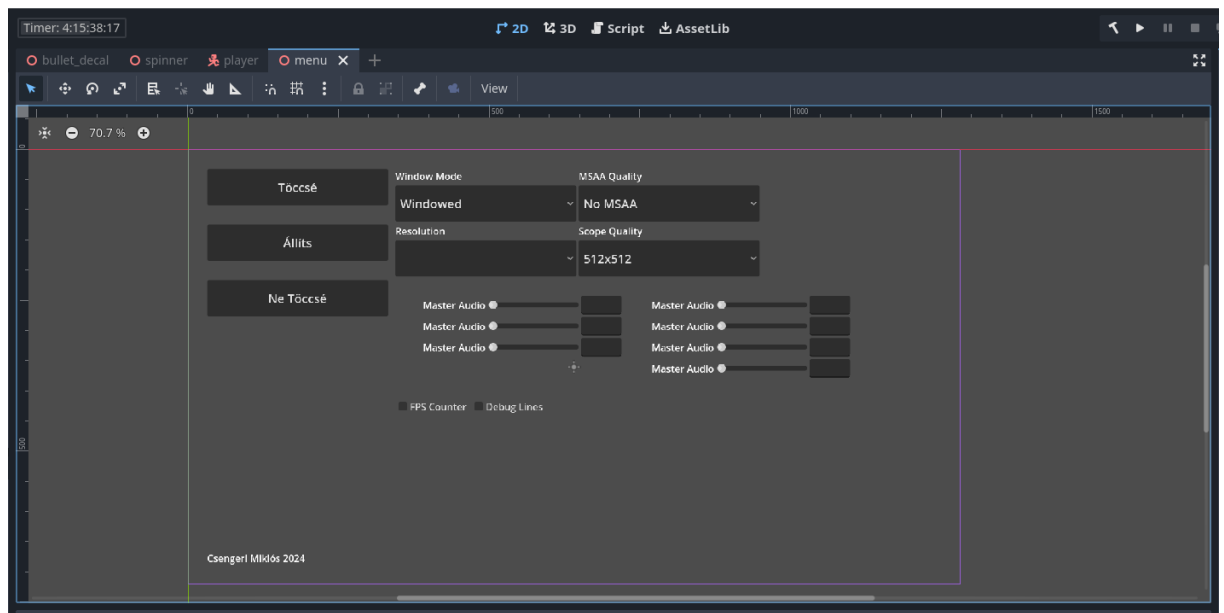
Azt hogy miért pont ezt választottam arról még mélyebben beszélek. De előtte itt van néhány kép a kinézetéről.



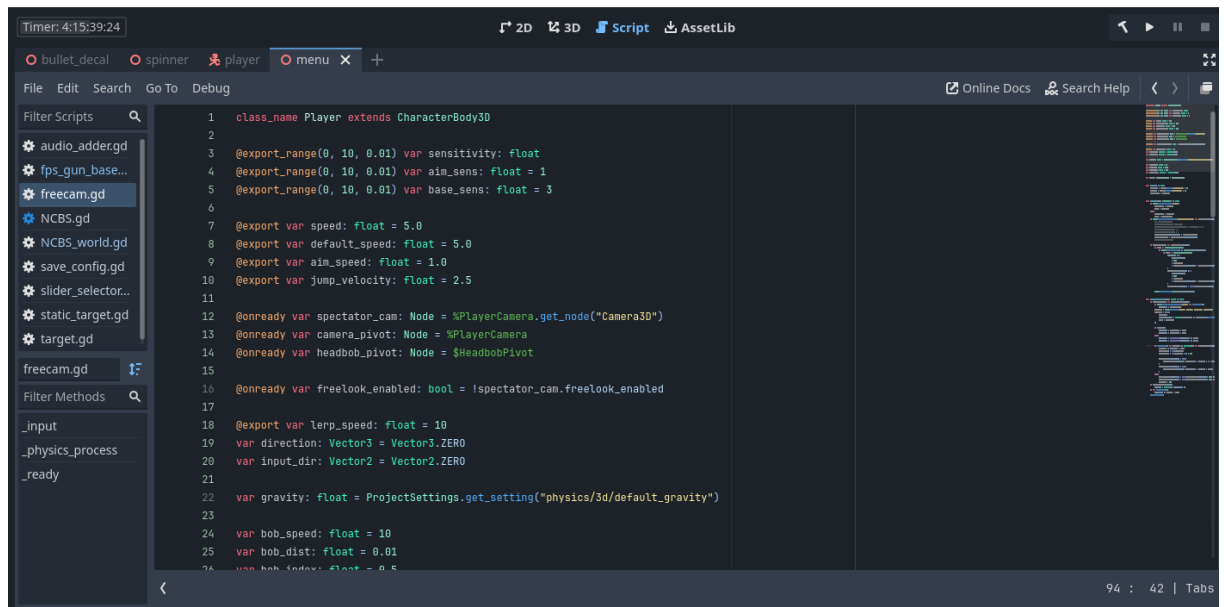
1. ábra Godot Project Manager



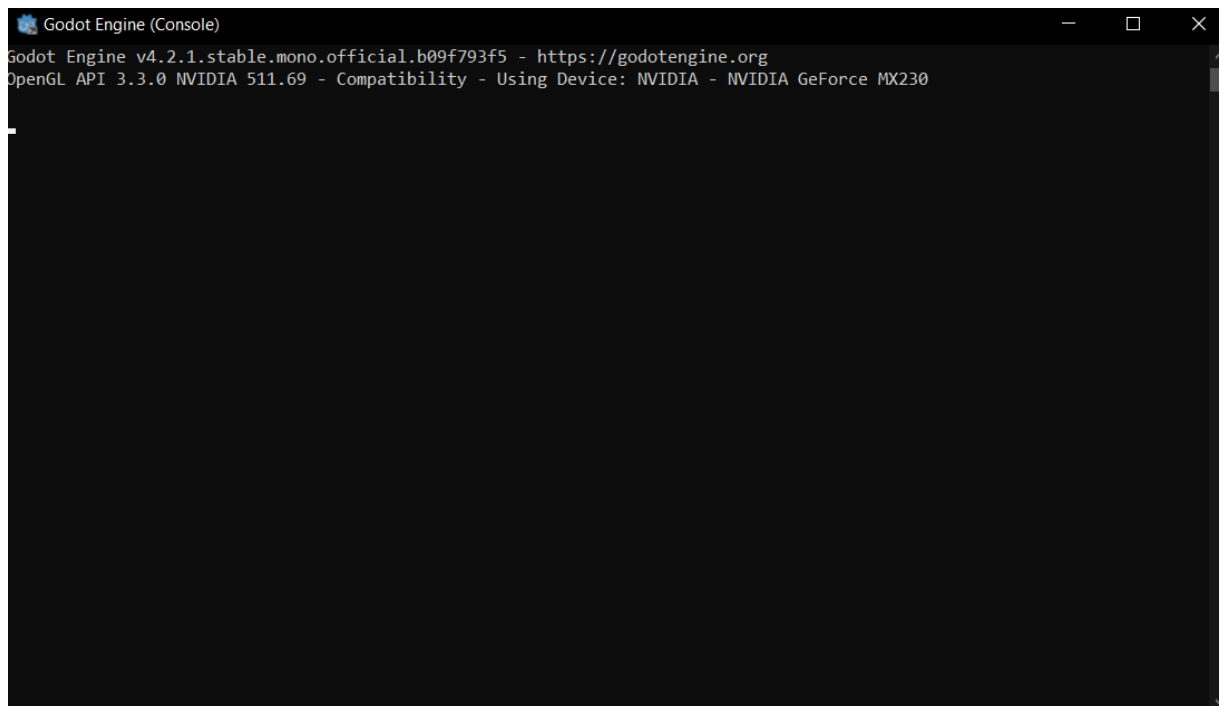
2. ábra Godot Editorja 3Ds nézetben.



3. ábra Godot Editorja 2Ds nézetben.



4. ábra Godot beépített szövegszerkesztője.



5. ábra Godot konzol ablaka.

Mivel Unity a legtöbbet használt játékmotor és én is használtam korábban ezért ahhoz fogom hasonlítani a Godotot. Maga a Godot is Godotban készült azaz minden ami a képeken van Godotba lett írva (kivéve a konzol az mivel az operációs rendszered alap konzolját használja) így mind a Project Manager mind maga a Játékmotor fő szerkesztője azon belül minden panel Godoton belül csinált. Így Unityvel ellentétben ahol van egy külön Unity Hub, Unity Editor és mellette egy szövegszerkesztő kényelmesebb használni. Maga a játékmotor szimplán gyorsabb Unityhez képest. Gyorsabban indul, fut, buildel és kisebb is. Míg egy teljes Unity installáció akár 15 GB-ot is elér Godot csak 150 MB kicsomagolva.

GodotSharp	2023. 12. 11. 23:47	Fájlmappa	
Godot_v4.2.1-stable_mono_win64.exe	2023. 12. 12. 11:07	Alkalmazás	106 092 KB
Godot_v4.2.1-stable_mono_win64_consol...	2023. 12. 12. 11:07	Alkalmazás	192 KB

6. ábra Godot Motor fájlrendszere

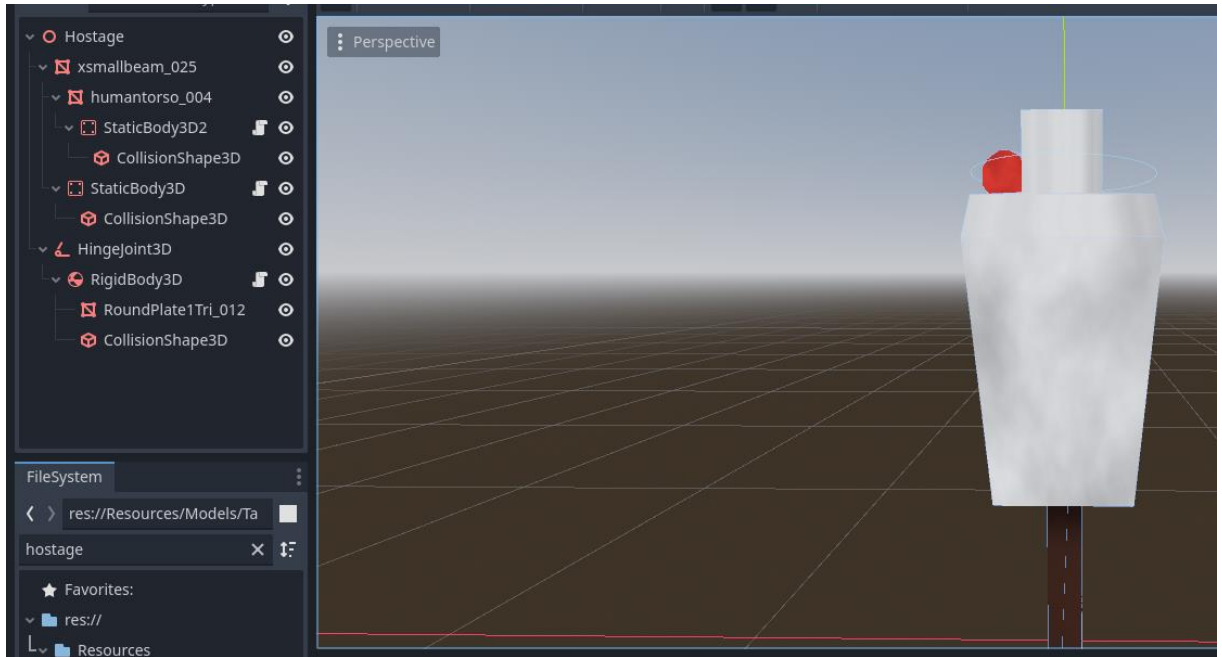
Projectméretek is jóval kisebbek a teljes kész projectem Godotban 400MB alatt van maga a játék meg kb. 200MB. Míg Unityben egy teljesen üres project 1.5GB-ot foglal el.

Funkcionalításban 2 dolgot tudnák kiemelni.

- Míg Unityben egy 2D project is 3D valójában Godotban a 2D és a 3D sokkal jobban szét van választva ezért is jobb 2D játékokhoz.
- Az egy hátránya Godotnak hogy az Editor nem frissül vizuálisan játék közben Unity-vel ellentétben, ezért nehezebb debugolni Unityhez képest néhány esetben. Így editorból csak

szövegesen a Node tulajdonságai alapján vagyunk képesek látni és változtatni az adatokat.

Nodeokról beszélve Godotban minden egy Node vagy Resource(picit túlszimplifikálva) A Nodeokat Scenekben helyezzük el és képesek vagyunk nekik adatokat adni amik Resourceok is lehetnek.



7. ábra Egy Godot Scene kompozíciója

A képen látható hogy Unityvel ellentétben ahol létrehoznánk egy GameObjectet és neki scripteket és különböző komponenseket adnánk itt hierarchikusan létrehozzuk a Sceneünket.

Ennek egyetlen limitációja hogy egy Node csak egy scriptet tartalmazhat. Ez viszont nem probléma mivel Godot erősen priorizálja a kompozíciót öröklődés helyett.

Emellett a Nodeokat egy Scenebe menthetjük el ami hasonló Unityhez viszont míg Unityben külön van Scene és Prefab itt minden egy Scene.

Resourceok adatokat tárolnak amit más Scriptek képesek használni ez Unityben hasonló a ScriptableObjecthez viszont Godotban 2 verziója is van a .tres(szöveges) és a .res(bináris)

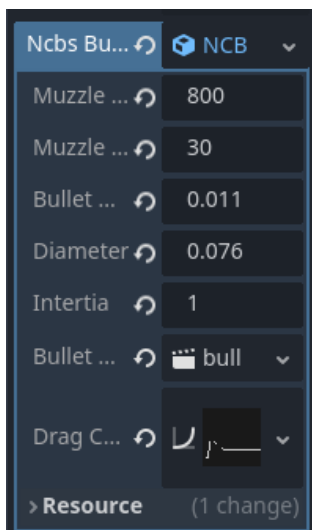
```

using Godot;
namespace NCBS
{
    [GlobalClass]
    29 references
    public partial class NCBSBulletRes : Resource
    {
        [Export]
        7 references
        public float MuzzleVelocity { get; set; }
        [Export]
        6 references
        public float MuzzleTwist { get; set; }
        [Export]
        9 references
        public float BulletMass { get; set; }
        [Export]
        7 references
        public float Diameter { get; set; }
        [Export]
        6 references
        public float Intertia { get; set; }
        [Export]
        8 references
        public PackedScene BulletMesh { get; set; }
        [Export]
        7 references
        public Curve DragCurve { get; set; }

        2 references
        public NCBSBulletRes()
        {
            MuzzleVelocity = 0f;
            MuzzleTwist = 0f;
            BulletMass = 0f;
            Diameter = 0f;
            Intertia = 0f;
            BulletMesh = null;
            DragCurve = null;
        }
        0 references
        public NCBSBulletRes(float muzzle_velocity, float muzzle_twist, float bullet_mass, float diameter, float inertia, PackedScene bullet_mesh, Curve drag_curve)
        {
            MuzzleVelocity = muzzle_velocity;
            MuzzleTwist = muzzle_twist;
            BulletMass = bullet_mass;
            Diameter = diameter;
            Intertia = inertia;
            BulletMesh = bullet_mesh;
            DragCurve = drag_curve;
        }
    }
}

```

8. ábra Egy C#-ban írt Resource



9. ábra Az a Resource a Godot Editorban

Ahogy a képen látható a 8. ábrán egy C#-ban írt Resource található amit a 9. ábrán egy GDScriptben írt script használ. A két nyelv között majdnem mindent lehet csinálni. Azzal az egy kivétellel hogy egy GDScript osztály nem öröklődhet C# Osztályból se vissza. De adatokat módosítani, Propertyket használni, Metódusokat meghívni szabadon lehet a két nyelv között.

Godot szerintem egy teljesen jó játékmotor viszont néhány dolog jelenleg hiányzik belőle de a következő kategóriában erről még beszélek mivel majdnem minden hibáját egy Addonnal orvosolni lehet. Ezek lehetnek akármik és mivel a Godot annyira nyitott akár új ablakokat is könnyedén hozzáadhatnak Godothoz például egy dialógus rendszer fület vagy egy vizuális scriptelés fület. Emellett saját Nodeokat is képesek hozzáadni amit én is használtam.

Godot Játékmotorhoz letöltött erőforrások

A motorhoz számos Assetet töltöttem le ezekből a fontosabbakat először felsorolnám.



A Godot alap 3Ds játékmotorja a [Bullet](#) motorból fejlődött ki viszont manapság már elavult és se nem determinisztikus, se nem gyors, se nem pontos és nagy játékokhoz használhatatlan. Ehhez képest a [Jolt](#) egy C++-ban írt fizikai motor ami egy személyes projectjeként indult de nagyon jó fizikai motorrá vált és még a [Horizon Forbidden West](#) című 200+ millió dolláros AAA játékban is használva volt. A [Godot Jolt](#) ezt a fizikai motort rakja bele Godotba és lehetséges hogy nemsokára le fogja váltani a Godot alap fizikai motorját. [Github](#)

A Jolt előnye sok más fizikai motorral szemben hogy direkt Open World (nyitott világú) játékokhoz fejlesztették ki. Így minden fizika által hatott Node könnyedén elalszik ha nem mozog ezzel erőforrásokat spórolva.

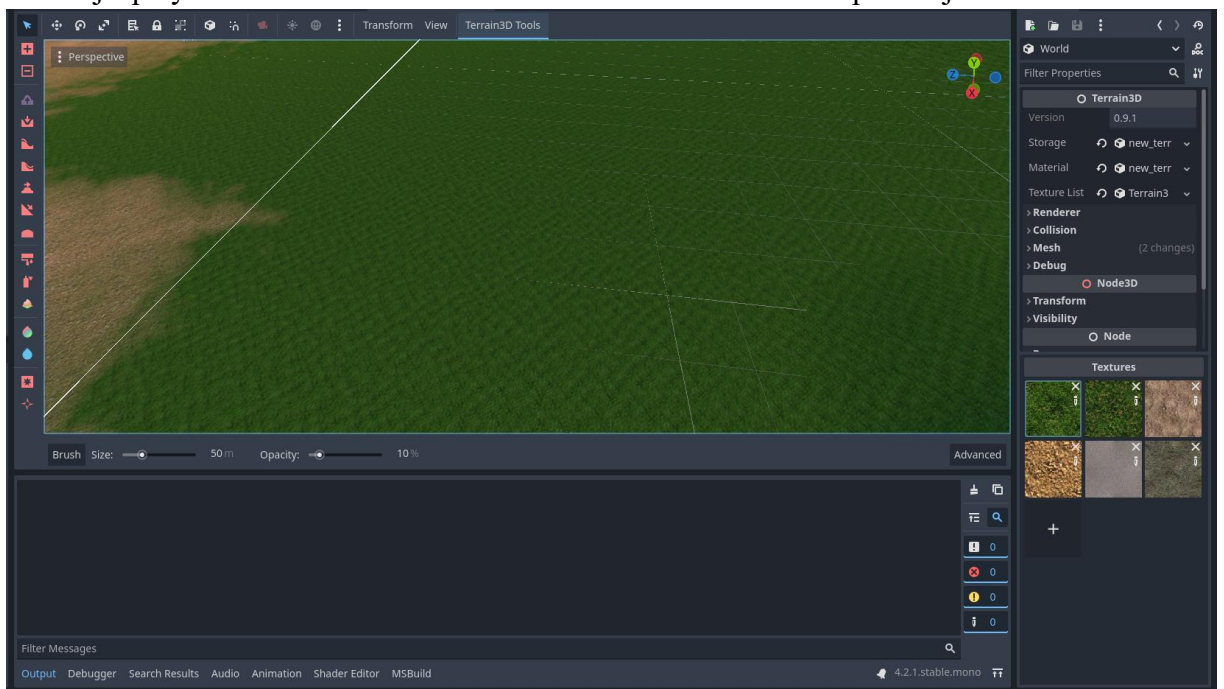
Nekem ez főként a célpontoknál kellet mivel az alap fizika nem volt se elég pontos se elég konzisztens hogy a célpontok jól reagáljanak.



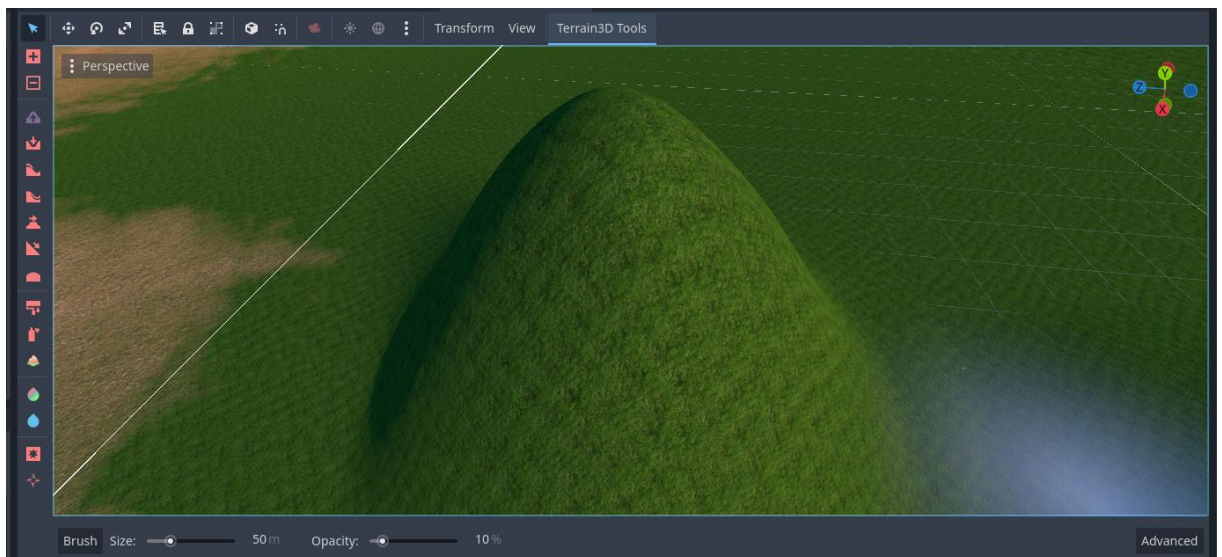
A [Terrain3D](#) egy Godothoz C++-ban írt környezet készítő Addon.

Mivel Godotban még nincs erre beépített megoldás [GitHub Issue](#) [GitHub Proposals](#) még ezért számos opcióból választhattam. A HTerrain, MTerrain és a Terrain3D. Eredetileg a MTerrain-t választottam mivel a HTerrain már jó ideje nem kapott frissítést és a sokkal lassabb GDScript nyelvben lett írva C++ helyett. Viszont az MTerrain míg papíron jobb mint a Terrain3D használatában sokkal könnyebb ezért átváltottam rá. A Terrain3D ugyanazt a rendszert használja mint a [Witcher 3](#) ami szintén egy nagy AAA játék.

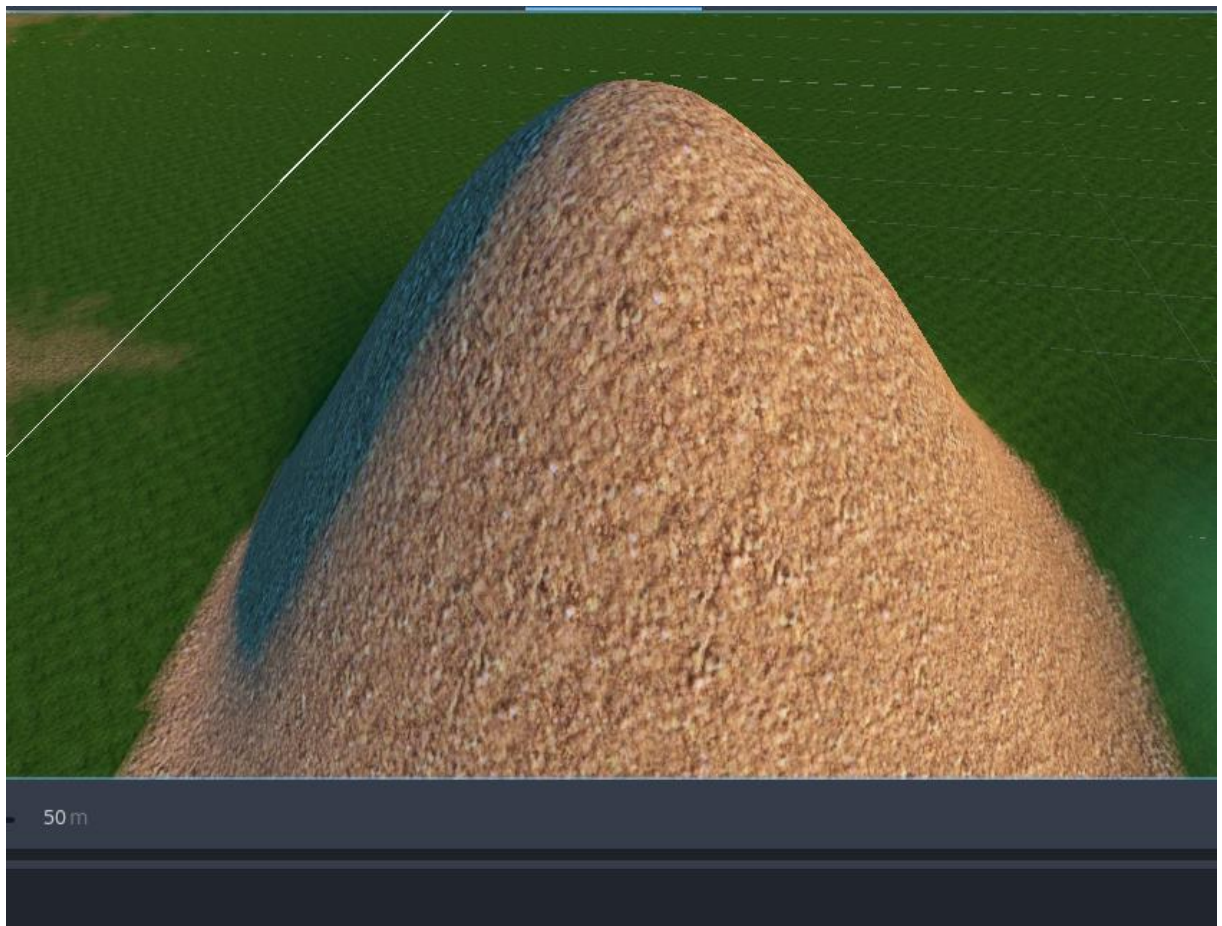
Ezzel könnyen lehet a pálya földjén változtatni és textúrázni. Így a nagy 6 kilométer átmérőjű pálya elkészítése nem volt túl nehéz. Mivel csak szimplán rajzolnom kellett.



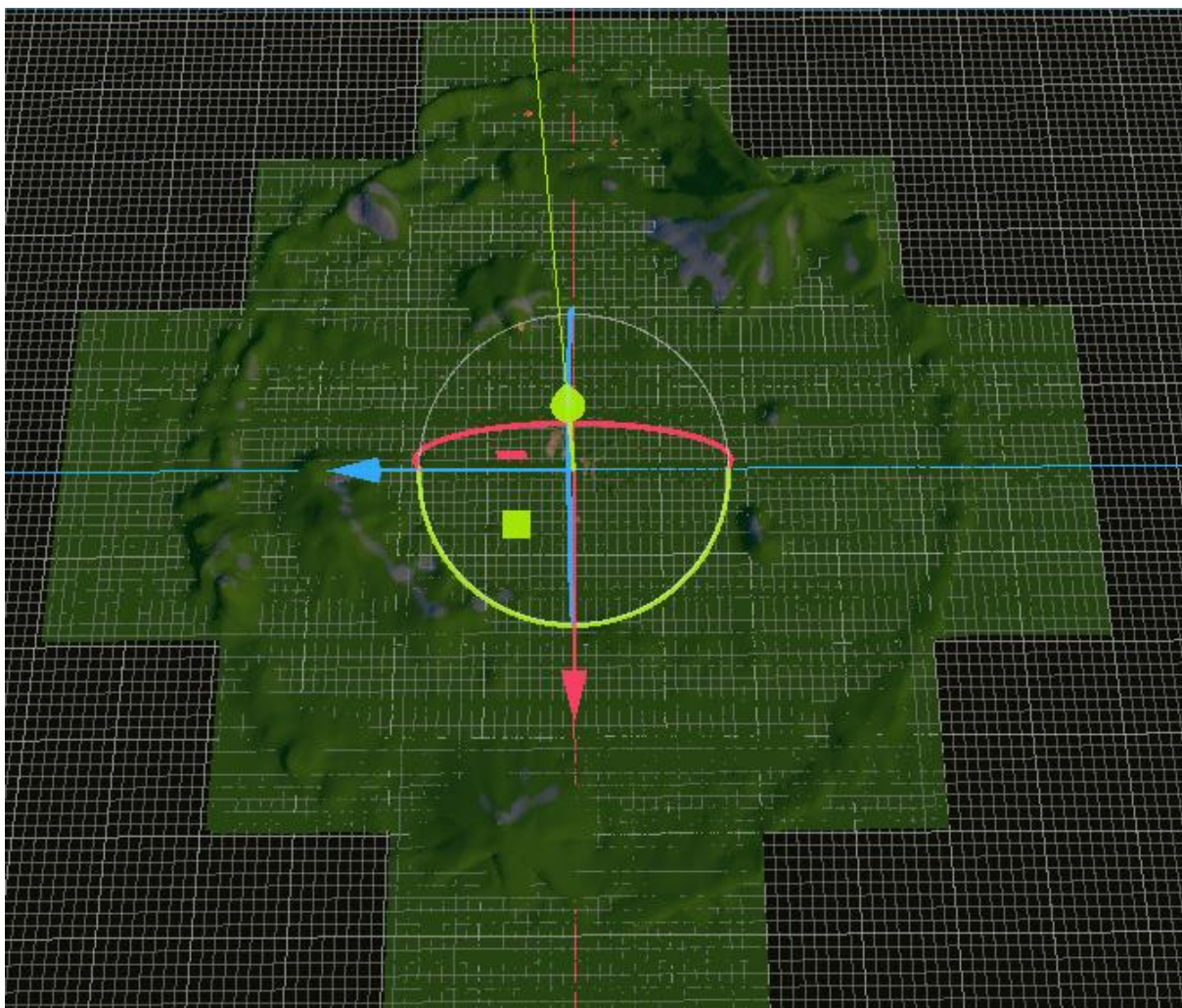
10. ábra Terrain3D palettája



11. ábra Terrain felemelése



12. ábra Terrain textúrájának változtatása



13. ábra A teljes világ

Ezek mellet más Addonokat is használok de mindegyiket ki lehetne törölni mivel nem adnak funkciókat Godothoz amik hatással vannak a játékra.

Ezeknek a listája.

- [Little Camera Preview](#) Ezzel képes vagyok kamerákat egy kis ablakba betenni hogy az editorban beállítsam őket.

- [Format on Save](#) Ez a Pythonban írt [GDScript Toolkit](#) formázóját hívja meg mentéskor.

- [Simple Timer](#) Ez arra van hogy számolja mennyi időt töltöttem a projecttel viszont nem használom az eleje óta és egyszer le is lett nullázva.

Voltak más Addonok is de vagy sosem használtam, vagy leváltottam.

Visual Studio Code

A [Visual Studio Code](#) avagy VSCode egy kódszerkesztő program amiben a Microsoft saját C# Extensioneivel és a [godot-tools](#) Extensionnel írtam a program C# részét. Lehetséges Godoton belül is C# kódot írni de egy külső editorhoz képest a képességei sokkal rosszabbak. VSCodeban a jobb debugolás és kódkiegészítés nagy segítség fejlesztésben. Míg én a sima verziót használtam ami nem nyílt forráskódú az alap forráskódja fent van [GitHubon](#) és van dedikált project mint [VSCodium](#) ami abból épít egy majdnem ugyanolyan verziót.

GDScript

A GDScript Godot saját programozási nyelve gyengén típusos, syntaxban és sok másban is Pythonhoz hasonlítható. A Godot játékmotorhoz nagyon szorosan kötött és mint a Python C++-t használ nagy erőforrásigényű metódusok futtatására a háttérben.

A nagy előnye hogy szimpla és gyors fejleszteni rá. Míg gyengén típusos a motoron belül akár megköveteljük hogy legyen statikusan írva ami felgyorsítja a kód futását.

Az egyetlen előnye a sebesség a játékmotorral való kommunikálásban míg például egy Node változóit nagyon gyorsan képes változtatni és olvasni a szoros integráció miatt minden másnak a sebessége C#-ban és még inkább C++-ban gyorsabb.

A játék kódjának legnagyobb részét GDScriptben írtam mivel szimpla dolgokra teljesen megfelelő. De ha amit szeretnénk írni aminek az erőforrás igénye nagy akkor jobban tesszük másban írni.

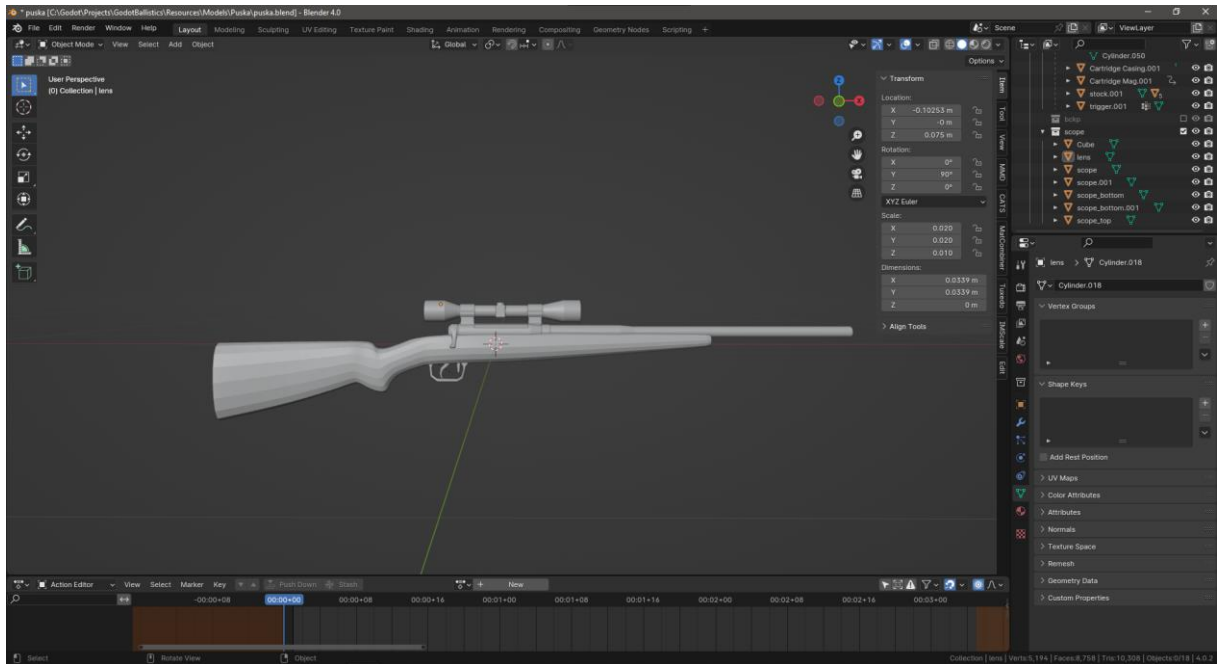
C#

A C# egy erősen típusos C-s syntaxot öröklő nyelv amit Microsoft a Java sikere miatt csinált. Mivel én Godothoz használtam ezért ahhoz hasonlítom. GDScriptel ellentétben nem interpreteres hanem compileres így a programokat futtatás előtt lefordítja ennek nagy előnye a sebesség. A C#-ban írt kód sebessége gyorsabb mint a GDScript majdnem minden esetben. Viszont a C++ mögött van. A C# kód és GDScript kód között nincs semmi valódi különbség. Így Godothoz ha kell utólagosan majdnem sorról sorra át lehet alakítani az egyikből a másikba a scriptünket majdnem minden esetben. Kivéve ha olyan dolgokat használunk ami C#-ban van de GDScriptbe nincs. Ilyen például az Interface.

Blender

A [Blender](#) egy nyílt forráskódú 3D modellező szoftver amit számtalan professzionális modellező használ és piacvezető is. Minden más alternatíva vagy rosszabb vagy drága. A játékban minden modelt ebben csináltam és a .blend fájlt a modell mappájában meg lehet találni emellett a puskának az animációja is itt készült. Míg nem a legkönnyebb megtanulni ha valaki egy 20-30 órát tölt vele már majdnem akármit képes modellezni használható minőségben. Emellett a Blender nem csak modellezésre és animálásra

alkalmas, lehet benne mást is csinálni például sok film is készült benne vagy másik példának Photo realiztikus képek is vannak.



14. ábra A puska modellje Blenderben

Audacity

Az [Audacity](#) egy nyílt forráskódú hangfelvétel és editáló szoftver a játékhoz használt hangeffekteket ezzel szerkesztettem.

GitHub

A [GitHub](#) egy verziókövető rendszer ami egy projecten való összedolgozást tesz kényelmesebbé. Én főként arra használtam hogy a laptopom és az asztali számítógépem között a projectet kényelmesen tudjam fejleszteni. Biztonsági mentésként is szolgált.

Obsidian

Az [Obsidian](#) egy fejlesztett jegyzetkönyv tele mindennel. Mappák, linkek, táblázatok akármi amit szeretnél. Én papírok gyűjtésére használtam főleg de utólag felesleges lett. Ha több dolgon kéne dolgoznom akkor hasznosabb lenne. De akármi amit a projecttel kapcsolatosan oda írnék azt GitHubba is beírhatom vagy egyenesen a projectbe egy script kommentjeként.

Az Obsidian egy Vaultban tárol adatokat és a Vault megtalálható a project törzsmappájában. .obsidian néven.

A Godot problémái.

A fejlesztés folyamata során számos buggal azaz nem kívánatos működésű dologgal találkoztam amiből párat kiemelek.

A távcső egy másik kamerával egy SubViewport-ot csinál, ami egy új ablakként funkcionál. Ennek a kamerának a képét a SubViewport-ból a távcső lencséjére rajzolom egy erre készített Viewport textúraként viszont futáskor ez egy hibát dob mindig.

```
ERROR: Viewport Texture must be set to use it.  
at: get_height (scene/main/viewport.cpp:137)  
ERROR: Viewport Texture must be set to use it.  
at: get_width (scene/main/viewport.cpp:127)
```

15. ábra Godot Consoleban lévő hibaüzenet

Ez egy játékmotorból eredő hiba amire van megoldás azzal hogy megvárjuk a Scene betöltését és utána adjuk hozzá kódból a textúrát. Mivel a hiba nem okoz semmi hibát játékon belül ezért nem javítottam ki. Jelenleg a játékmotor GitHub-ján egy nem megoldott Issue-ban szerepel és megjavításra vár. Ezt valószínűleg a Godot 4.3.1-be fogják megjavítani emellett a 4.2.2-be amikor az megjelenik. [Github Issue](#)

Következőnek míg ez minden játékmotorra jellemző a Z-Fighting is okozhat problémát ami akkor történik ha 2 Mesh ugyanazon a Z-Bufferen lesz lerajzolva és utána grafikai hibák jelennek meg ahol nem kéne látni a hátsó objektumot. Ez nagy távon történik meg főként.

A képen direkt előhoztam a hibát különben ilyen relatív rövid távon nem jelenne meg.



16. ábra Z-Fighting

Ezt úgy lehet minimalizálni, hogy a camera Clipping Plane-eit közelebb hozzuk egymáshoz főként a közelit növeljük. Ez a kamera minimális és maximális látótávolságát adja meg.

Emellett a modellekben is csökkenthetjük azzal hogy nem rakunk több vékony elemet közel egymás mögé.

Ezekkel csak annyit érünk el hogy nagyobb táv kell a kamerától vagy kisebb táv 2 Mesh között míg megint előjön ez a hiba.

Viszont végleges megoldás már van Godotban és a következő verziótól benne is lesz! [Github Issue](#) és szintén [Pull Request](#).

Felhasználói Dokumentáció

A játék egy Sandbox FPS magyarul egy belső nézetes lövöldözős játék amiben szabadon csinálhatunk akármit amit a játék enged háttértörténet és story nélkül.

Számos reaktív célpontra képesek vagyunk lőni amik másképp reagálnak.

Rendszerkövetelmények

A Godot játékmotor miatt a rendszerkövetelmények alacsonyak viszont mivel 3Ds játék ezért egy videokártya nagyon ajánlott.

A játék 60 képkockával fut másodpercenként ha ezt nem vagyunk képesek elérni akkor hibákat tapasztalhatunk. Ha ezt látni szeretnénk akkor az FPS Countert bekapcsolhatjuk.

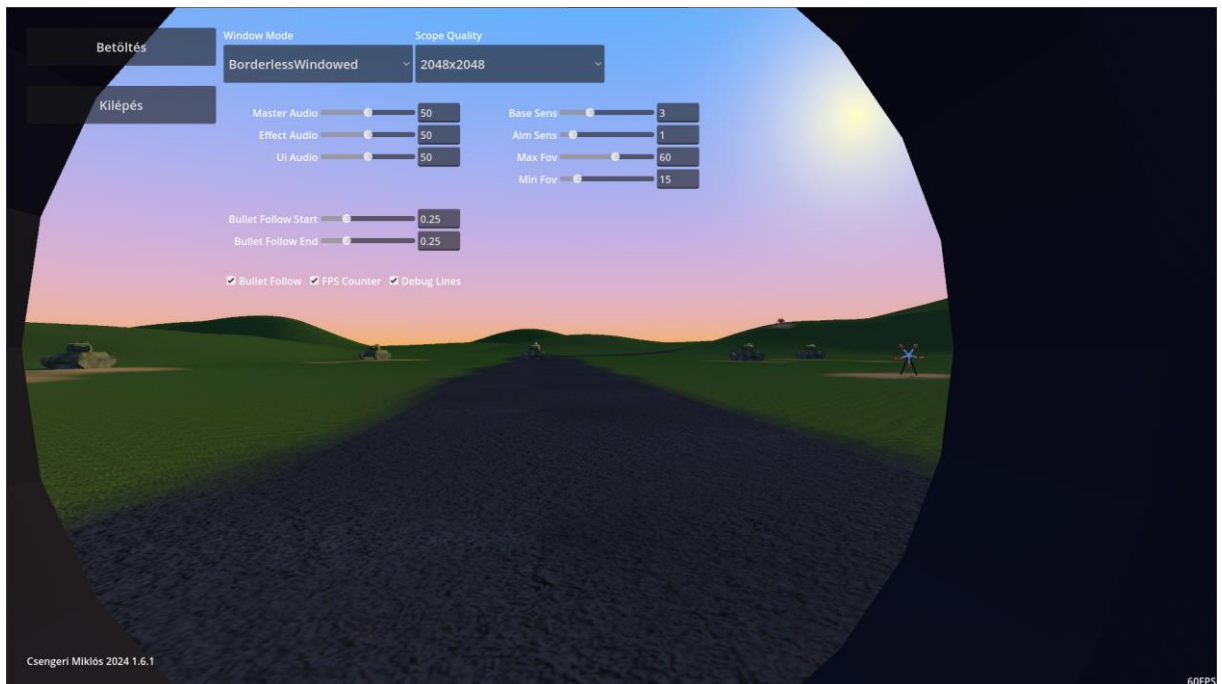
Minimum Gépigény

- Operációs rendszer: Windows 7,8/8.1,10
- Processzor: 4 magos Intel vagy AMD laptop processzor.
- Memória: 4GB DDR3
- Videokártya: Dedikált videokártya 2GB Memóriával Vulkan supportal.
- Tárhely: 500MB

Ajánlott Gépigény

- Operációs rendszer: Windows 7,8/8.1,10
- Processzor: 6 magos Ryzen 5 processzor vagy azzal egyenlő Intel
- Memória: 8GB DDR4
- Videokártya: Nvidia GTX 1660 6GB
- Tárhely: 500MB

Menü



17. ábra A menü

A menüben számos dolgot be tudunk állítani.

- A Betöltés gombbal el tudjuk indítani a játékot ha már bent vagyunk a játékban az ESC gombbal vissza tudunk lépni a menübe.
- A Kilépés gomb megnyomásakor a játék bezárja magát.
- A Window Mode-nál be tudjuk állítani a játékot teljes képernyőre vagy ablakos módra. Erre 3 opciónk van. A sima ablakos azaz Windowed az alap mód és itt a felbontást is képesek vagyunk állítani. A BorderlessWindowed és a ExclusiveFullscreen teljes képernyős módok és teljesítmény érdekében az ExclusiveFullscreen ajánlott. Különben a BorderlessWindowed de majdnem ugyanaz a kettő látszatra.
- A Scope Quality a távcsövek minőségét állítsa be képpontokban. Teljesítmény érdekében lehet csökkenteni.
- Alattuk van 3 hang Slider amivel a játék hangerejét tudjuk állítani. A Master Audio minden hangot. Az Effect Audio a hangeffektek hangját és végül a UI Audio a játék felületének a hangjait állítja.
- A Következő 4 Slider jobbra az érzékenység és kamera szögének a limitjeit állítja. A Base Sens az alap érzékenység míg a Scope Sens a távcső használata közben van használva.
- A Max és Min Fov a játékos kamerájának a maximális és minimális értékét állítja, amit játékon belül a görgővel tudunk változtatni.
- A Bullet Follow beállításai a golyókövetés kezdési ideje másodpercben (Bullet Follow Start) és a találat utáni extra időt (Bullet Follow End) változtatja.
- Végül van 3 CheckBox amiből az első a Bullet Follow rendszer alap módját változtatja. (Alapból be van e kapcsolva vagy nem) A második a képernyő jobb alsó sarkában lévő FPS(képkocka másodpercenként) számlálót engedélyezi. A harmadik pedig a golyó útját követő kék vonalat rajzol emellett hogy jobban látszódjon az útja.

Mozgás és kamera

A karakterünket a WASD billentyűkkel tudjuk mozgatni mint a legtöbb játékban.

A Shift letartásával futni is tudunk.

A kameránkat az egérrel képesek vagyunk elforgatni az érzékenységet a menüben képesek vagyunk állítani.

A kamera látásszögét(FOV) a görgővel tudjuk változtatni ennek a limitjeit szintén a menüben találjuk. Célzáskor ajánlott csökkenteni a látásszöget hogy nagyobb képünk legyen.

Az alap nézetünk mellet van egy szabad kameránk amit az E gomb megnyomásával tudunk elérni és ebben a kamerában szabadon tudunk repülni. Ebben a módban a Shift+görgő használatával képesek vagyunk felgyorsítani a repülésünket. Ha vissza szeretnénk menni azt a Q gomb megnyomásával tudjuk elérni.

Az Alt+görgő használatával képesek vagyunk az időt lassítani viszont ez nem ajánlott.

Lövés és célzás

A bal klikk megnyomására a puskánkat el tudjuk sütni és ha a gyolyókövető kamera be van kapcsolva akkor automatikusan át fog váltani.

A gyolyókövető kamerát a V gombbal vagyunk képesek ki-be kapcsolni.

A jobb klikk letartásával képesek vagyunk a távcsövet használni hogy pontosan tudjunk távolra löni.

Fejlesztői dokumentáció

A játékban nagy mennyiségű script található. 4 script fajtát különböztetnek meg az első a sima GDScript script ezeket a fájlkezelőben meg lehet találni. A második az beépített GDScript ezek a Scene-be vannak benne így nem jelennek meg a fájlkezelőben ezeket gyakori használni olyan scriptek esetében amik vagy csak egyszer használatosak vagy olyan szimplák hogy nem érdemes a mappáinkat telerakni velük habár ennek a vonala hogy mikor melyiket használjuk nem fix. A harmadik fajta az egy Autoload script ezek annyiban speciálisak hogy a játék indításakor létre lesznek hozva és a Root-hoz hozzáadva ezek a Singleton vagy Managerekre hasonlóak és programozási nyelvük akármi lehet. Végül a C# scriptek és mivel a ballisztika teljes rendszere C#-ban van írva ezért csak az van ott.

Még vannak Resourceok amik scriptként vannak elmentve viszont adatokat tárolnak. Ezeket az adatokat mentésére és editorból könnyű beállítására kényelmes használni.

Name	Path	Global variable				
FollowObjectManager	res://Resources/Scripts/Camera/game_camer...	☒ Enable	📁	⬆	⬇	🗑
TimeManager	res://Resources/Scripts/Utility/time_manager...	☒ Enable	📁	⬆	⬇	🗑
SettingsManager	res://Resources/Scripts/Utility/settings_mana...	☒ Enable	📁	⬆	⬇	🗑
DebugTools	res://Resources/Scripts/Utility/debug_tools.gd	☒ Enable	📁	⬆	⬇	🗑
AudioMixer	res://Resources/Scripts/Utility/audio_mixer.gd	☒ Enable	📁	⬆	⬇	🗑
NCBS	res://addons/NCBS/NCBS.cs	☒ Enable	📁	⬆	⬇	🗑

18. ábra Az Autoloadok listája

Először az Autoloadok. Míg ezek sima scriptek speciálisan vannak használva.

Az első a FollowObjectManager:

Ez a játék összes kamerájáért felelős benne a takarítás és kameraváltás. Minden kamerának van egy ID-je ami egyedi.

2 Signalt tartalmaz: use_camera amivel kamerát lehet választani és clear_cameras amivel a nem létező kamerákat kitörli.

TimeManager

Ez a játék idejét kezeli és csak más scriptekből vannak a funkciói meghívva.

2 ilyen funkciója van az első a set_time_scale(scale: float) amivel a motor time scalejét tudjuk beállítani 1-től 0.015-szörös ideig.

Másik a reset_time_scale() ez meg visszateszi az eredeti értékekre az időlassítást.

SettingsManager

Ez a játék beállításaiért felel. Ennek is 2 fő funkciója van a save_var(setting_name: String, value:) ami elment egy beállítást és a get_var(setting_name: String) ami meg visszaadja egy beállítás értékét.

A játék összes beállítását egy save_config.gd nevű Resourceban mentem el amiben egy string:var Dictionary-ben van az összes beállítás. Emellett benne van az alap mentés és betöltés. Emellett egy config.tres fájlt létrehoz a játék gyökérmappájában amiben ezek az értékek találhatóak.

DebugTools

Ezzel képesek vagyunk egy sorral 3Ds vonalakat és gömböket rajzolni mivel alaphoz Godotban csak 2Dhez vannak ilyen funkciók. A Debug Lines beállítás is ezzel rajzolja a vonalakat.

AudioMixer

Ennek annyi a funkciója hogy akárhonnán egy hangbusz hangerejét megváltoztassuk a `change_audio_level(mixer_name: String, audiolevel: float)` funkcióval.

NCBS

Ez felelős a játék ballisztikájáért és a saját Godot Addonomként működik így csak ezt magában képes lennék kirakni hogy más is tudja használni. Majd a C# résznél mélyebben belemegyek.

Másodszor a Built in azaz beépített scriptek mivel ezek általában szimplábbak de nehezebb megkeresni ezeket mivel a Scenekhez vannak kötve.

Menu:

A Menu a menüért felel. A SettingsManagerből betölti és beállítja az adatokat. Emellett sok kis dologért felel például a játék betöltése.

BasicVersionNamer:

Ez csak annyit csinál hogy a játék Project beállításaiából kiszedi a verziót és a bal alsó sarokban a nevem mellett kiírja

Sway:

Ez a menüben a 3Ds kamerát forgatja

tank_cannon_angle:

Ez a tankoknak a csövét és a lövegtoronyát forgatja.

A Sima GDScript Sciptek

pause_menu.gd:

Ez betölti a menüt ha megnyomjuk az Esc gombot.

Ez csak azért nem Built in mivel másik pályánál is lehet használni.

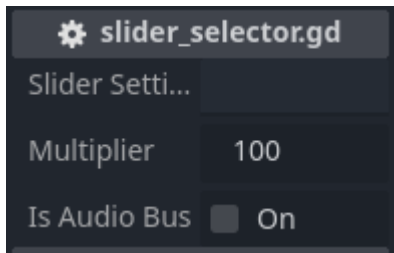
slider_selector.gd:

Ez a menüben a Sliderekért felelős



19. ábra egy Slider

Szinkronizálja az értékeket, beállítja a szöveget



20. ábra A Slider beállításai

Most 3 egyben

static_target.gd

target.gd

target_destructible.gd

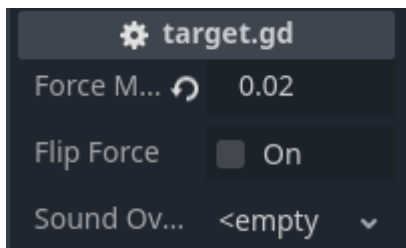
Mindhárom a célpontokért felelős annyi a különbség hogy a statikus az statikus colliderekhez van és a destructible az meg letörli magát ha eltalálják.

Mindhárom fő funkciója a handle_hit(bullet: NCBSHitRes)

Ez a saját NCBS rendszeremből egy golyó találati adatait kezeli.

Lejátssza a hangot és hozzáadja a találati helyhez a Decalt. Emellett be lehet állítani az erő szorzóját, megfordítani az erőt és még a hangot is felülírni.

Ezeket újra kell írni mivel se az öröklődéshez se a kompozíciós módszerhez nem tartoznak. Viszont legalább 2 script úgy is kellene mivel a static_target.gd az statikus colliderhez van írva míg a másik 2 RigidBody alapú.



21. ábra Egy target beállításai

camera_bullet.gd:

Ez a Bullet Follow rendszerért felelős. Átállítja a kamerát és vissza főként. Ez is a game_camera rendszerrel működik.

game_camera.gd

Ez minden kamera alapja.

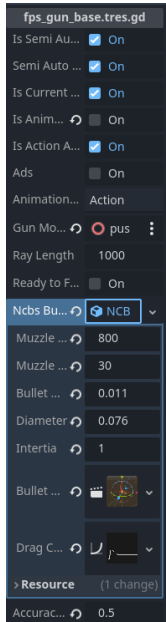
regisztrálja magát a FollowObjectManager-nek

spectator_camera.gd

Ez a játékos szabad kamerája csak szimplán lehet vele repülni.

Viszont inputért is felelős hogy képesek legyünk időt lassítani és zoomolni.

fps_gun_base.gd



22. ábra A puska beállításai

Ez az egyik legkomplexebb scriptem és egy fegyver elsütéséért és adataiért felelős.

Sok féle fegyvert be lehet vele állítani és létrehozza az NCBS rendszerhez a lövedéket.

Biztosítja hogy a fegyver csak akkor lőjön ha szabad neki.

Megnézi hogy a fegyver a mostani fegyverünk, a lövés animációja be van fejezve, más animáció is be van fejezve és ha fél automata fegyver akkor azt is megnézi hogy elengedtük az egér gombot és megint lenyomtuk.

Míg ez csak a puskánál van használva nagyon könnyen más fegyverhez is beállítható pár perc alatt.

Az NCBS rendszerhez egy NCBSBulletRes-t is tartalmaz.

audio_adder.gd

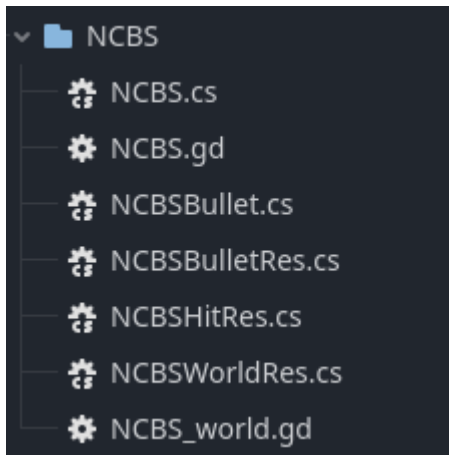
Ez a menüben a gombokhoz hozzáadja a hangokat.

Induláskor minden Nodeon átmegy és ha megfelel valamilyen feltételnek akkor hozzáadja a hangot amit megadtunk.

Képes akármilyen hangot adni de én csak menüben használom.

Ez minden script lehet van olyan amit véletlen kihagytam vagy az is lehet hogy csak nincsen használva.

És végül az NCBS rendszer ami egy Godot Addonként van implementálva és külön működik a projecttől.



23. ábra NCBS fájlrendszere

A komplexitása miatt először leírom melyik script mit csinál és utána egy golyónak az életét is leírom.

NCBS.cs:

A fő script ami a minden lövedék kezelésért felelős.

NCBS.gd

Ez automatikusan hozzáadja a NCBS.cs-t az Autoloadokhoz és betölti a saját Nodejaimat.

NCBS_world.gd:



24. ábra NCBS_world beállításai

Ez egy custom Node ami a világ fizikai tulajdonságait mondja meg mivel az NCBS rendszer nem használja az alap fizikát. Ezt még fejleszteni lehet például azzal hogy magasabban kisebb a légsűrűség. Ha nem használjuk az NCBS magától feltölti realisztikus adatokkal.

NCBSWorldRes.cs:

Egy Resource ami ezeket az adatokat tárolja.

NCBSBullet.cs:

Ez egy golyó és minden adatát eltárolja és ebben van a ballisztika kiszámításához is a funkció. Egy Queue-ben eltárolja ezeket a pozíciókat alaptól 1000szer másodpercenként.

NCBSBulletRes.cs:

Ez egy Resource ami a golyó adatait tárolja. Az egyetlen értéke ami nem egyértelmű az a Drag Curve ami egy Godot Curve ami a golyó légellenállási értékét(Cd) adja meg különböző mach számokon viszont mivel a Curve 0-1 között van ezért 10-el osztani kell így mach 1 az 0.1 és mach 2 az 0.2.

NCBSHitRes.cs:

Ez egy Resource ami specifikusan találkor az ahhoz fontos információkat tárolja. Ezeket a target scriptek kezelik a handle_hit(bullet: NCBSHitRes) metódusukkal.

Az egész rendszer nagyon állítható és könnyen bővíthető.

Ha magadnak szeretnél valami belerakni azt könnyen meg lehet tenni.

De alapból ha csak letöltöd képes vagy golyókat létrehozni 1 sor kóddal. Amik pontosabban vannak szimulálva mint majdnem akármilyen játék.

Minden kis dolgot le tudnék róla írni de még minden változhat vagy változik

Ezért az alap logikát írom le. Vagy másképp mondva egy NCBS golyó életét.

először létrehozzuk a golyót egy pozícióval és egy NCBSBulletRes-el

```
NCBS.AddBullet(add_innacuracy_moa(firepoint, accuracy_moa), NCBS_bullet)
```

25. ábra Golyó hozzáadása

ez a fő NCBS.cs-ben hozzáad egy új golyót a listájába

```
}  
public void AddBullet(Transform3D firepoint, NCBSBulletRes bulletres)  
{  
    >| Bullets.Add(new NCBSBullet(firepoint, bulletres, WorldRes));  
    >| debugLines = (bool)settings.Call("get_var", "debug_lines");  
}
```

26. ábra Golyó hozzáadás kódrészlete C#-ban

Utána minden Physics Frame közben átmegy az összes golyón. Ami alapból 60-szor fut le másodpercenként.

```
foreach (var bullet in Bullets)  
{  
    >| if (!bullet.Initialized)  
    >| {  
    >| >| Initialize(bullet);  
    >| >| Debug.Print("Bullet Initialized!");  
    >| }  
}
```

27. ábra Golyó inicializálása

És mivel még nem lett inicializálva ezért ezt megteesszük.

alap értékeket beállítunk az adatokból amit a létrehozáskor adtunk meg.

```
public void Initialize(NCBSBullet bullet)
{
>| PackedScene bulletscene = GD.Load<PackedScene>("addons/NCBS/bullet_node.tscn");
>| if (bullet.Data.BulletMesh != null)
>| {
>| >| bullet.BulletNode = bullet.Data.BulletMesh.Instantiate<Node3D>();
>| }
>| else
>| {
>| >| bullet.BulletNode = bulletscene.Instantiate<Node3D>();
>| }
>| bullet.BulletNode.Basis = bullet.StartPosition.Basis.Orthonormalized().Rotated(new Vector3(0,1,0),90);
>| GetTree().Root.AddChild(bullet.BulletNode);

>| bullet.LastDelta = 0;
>| bullet.StartPhysicsStep = Engine.GetPhysicsFrames();
>| bullet.CurrentVelocity = new Vector3(bullet.Data.MuzzleVelocity, 0, 0);
>| bullet.StartTime = CurrentTime;
>| bullet.CurrentPosition = bullet.StartPosition;
>| BulletState NewState = new BulletState(0, bullet.CurrentPosition.Origin, bullet.CurrentVelocity);
>| bullet.Positions.Enqueue(NewState);
>| GD.Print(NewState.Position.ToString() + " " + NewState.Velocity.ToString());
}
```

28. ábra Golyó inicializálásának kódrészlete

Emellett a vizuális részét is létrehozzuk.

Ezek után megnézzük hogy mennyi időt számítottunk előre a golyón és ha az túl kevés akkor vagy kezd fogyni akkor számolunk többet.

Minden golyó általában 3 fajta pozícióadatot tárol. Mind a múlt, jelen és a jövőbeli pozícióját is tárolja.

- Ami már elavult és csak ki lesz dobva
- Ami pont a mostani ami használva lesz
- Ami a jövőben van és még nem lett használva

Mivel még nincs neki adata ezért kiszámoljuk addig amíg nincs eléggé a jövőben ami után megtaláljuk a mostani pillanatot. Mindent korábban kidobunk.

```
do { bullet.CurrentState = bullet.Positions.Dequeue(); skipped++; }
while (bullet.CurrentState.Delta + bullet.StartTime < CurrentTime);
//if (skipped != 16) { GD.Print("Found the current one! Skipped: " + skipped); }
//Raycast!
Vector3 Start = bullet.LastPosition.Position;
Vector3 End = bullet.CurrentState.Position;
```

29. ábra Golyó jelenlegi pozíciójának a kiválasztása

Létrehozunk egy hitscan sugarat ami a golyó előző és mostani pozíciója között megy és ha az eltalált valamit akkor kategorizáljuk a találatok. 3 kategória van az első hogy a golyó valami okból nem releváns vagy nem is talált el semmit a második hogy eltalált valamit de amit eltalált annak nincsen egy handle_hit() funkciója ezért nem tudunk vele sokat csinálni. De ha van olyan funkciója akkor meghívjuk neki és vége.

```
Dictionary hit = Space.IntersectRay(PhysicsRayQueryParameters3D.Create(Start, End));
if (hit.Count != 0)
{
    >| bullet.HitRes = new NCBSHitRes(CurrentHitID, 1, hit, bullet.getJoules, new Transform3D(bullet.Basis, bullet.LastPosition.Position), bullet.Data)
    >| CurrentHitID++;
    >| if (debugLines)
    >| {
    >| >| debugTools.Call("draw_line_color", bullet.CurrentState.Position, hit["position"], Color.FromHtml("FF0000"));
    >| }

    >| bullet.BulletNode.Set("global_position", hit["position"]);
    >| Node3D hitnode = hit["collider"].Obj as Node3D;
    >| if (hitnode.HasMethod("handle_hit") && bullet.HitRes.HitCode != 0)
    >| {
    >| >| bullet.HitRes.HitCode = 3;
    >| >| hitnode.Call("handle_hit", bullet.HitRes);

    >| }
    >| bullet.HitRes.HitCode = 2;
    >| bullet.Clear();
    >| hit_pending = true;
    >| //Hits.Add(bullet);
    >| //Bullets.Remove(bullet);
    >| }
}
```

30. ábra Kódrészlet ami eldönti hogy a Golyó eltalált e valamit

Ha most pont nem találtunk el semmit akkor csak elmozgatjuk a golyónak a vizuális részét és elforgatjuk hogy jól álljon.

```
else
{
    >| if (debugLines)
    >| {
    >| >| debugTools.Call("draw_line_color", Start, End, Color.FromHtml("0000FF"));
    >| }

    >| //debugTools.Call("draw_line_color", bullet.CurrentState.Position, bullet.Positions.Peek());
    >| //debugTools.Call("draw_line", current.Position, bullet.LastPosition.Position);
    >| if (Start != End){
    >| bullet.BulletNode.LookAtFromPosition(Start, End);
    >| }

    >| bullet.LastPosition = bullet.CurrentState;
    >| bullet.BulletNode.Set("global_position", bullet.CurrentState.Position);
}
```

31. ábra Ha nem talált el semmit a golyó forgatása és mozgatása

Utána még csinálhatunk akármit itt például megnézem ha túl alacsonyan van de nem talált el semmit azaz ki lett löve a világból akkor csak töröljük ki.

```

>| if (bullet.CurrentState.Position.Y <= -100)
>| {
>| >| new NCBSHitRes(0, 0, null, 0, bullet.CurrentPosition, bullet.Data);
>| >| bullet.HitRes.HitCode = 1;
>| >| bullet.Clear();
>| >| hit_pending = true;
>| >| //Hits.Add(bullet);
>| >| //Bullets.Remove(bullet);
>| }

```

32. ábra Ha a Golyó kiesik a pályából kitöröljük.

Ha a golyónk eltalált valamit akkor ki is lesz törölve még a következő Physics frame előtt. Különben megy tovább előre az időben.

Van sok funkciója is a rendszernek példának itt van egy ami kiszámolja hány Joule erővel talál be a golyó magyarul milyen ereje van. Magyarországon 7.5 Joule a maximum legális engedély nélkül tartható légpuska meg más fegyver is. Ez a puska egy 7.62 NATO lövedék alapján van mintázva így közelre több mint 3000 Joule az ereje találakor.

```

3302.97338867188
At: res://Resources/Scripts/Ballistics/target.gd:54:joule_to_meters()

```

33. ábra A golyó ereje ≈ 50 méteren

Bullet mass/type	Velocity	Energy
147 gr (10 g) M80 FMJ	2,800 ft/s (850 m/s)	2,559 ft-lbf (3,470 J)
150.5 gr (10 g) M59 mild steel core FMJ	2,809 ft/s (856 m/s)	2,648 ft-lbf (3,590 J)
175 gr (11 g) M118 long range BTHP	2,600 ft/s (790 m/s)	2,627 ft-lbf (3,562 J)

34. ábra 7.62x51 NATO sebessége és energiája

Ami a [valódi adatokhoz](#) reális.

Viszont 400 méterre már csak 2200 Joule ereje marad.

```

2217.8056640625
At: res://Resources/Scripts/Ballistics/target.gd:54:joule_to_meters()

```

35. ábra A golyó ereje ≈ 400 méteren

A rendszer messze van a tökéletestől viszont így is használható. Az egészet újra kéne írnom és raknom is kéne bele különböző ballisztikai model opciókat. De a mostani is pontosabb mint majdnem minden videójáték. Sok dolog ami hiányzik belőle mint például a szél azt csak 3Ds model mellet tudnám pontosan belerakni és valószínűleg a világhoz menne a Resourceba.

Jelenleg az egyik probléma hogy nincs float alapú idő hanem a 60 FPS-hez lefixálva számol. Ez annyit jelent hogy az golyó számítási lépéseinek oszthatónak kell lennie 16-al. Azaz csak 1,2,4,8 és 16 lehet. Az 1ms az 1000 számítás másodpercenként a 16ms az

meg csak 60. Float alapú idővel az egész modelt pontosan le lehetne lassítani és utána a gyoló számítási lépése lehetne akármilyen szám akár 0.1ms vagy magasabb mint 16ms mint például 32ms vagy 37.13ms is. Funkcionalításban különben nem változna semmi mivel a lövedék ugyanúgy számolná ki magát. És ugye mivel 60 FPS az nem 16ms hanem 16.666ms ezért az egész model 4%-al lassabb a valós időnél viszont maga a számítás pontos, csak a megjelenítés lassú.

Asset források és licenszek

A játékban a hangeffektek a [FesliyanStudios](#)-tól vannak a menü hangjai egy [Bfxr](#) nevezetű szoftverrel lett generálva.

Az összes modelt Blenderben csináltam magamnak. A .blend fájlok is ott vannak a mappákban.

A világ textúrai innen vannak [Aidan Lawrence](#) (2024.03.15) Minden más textúrát magamnak csináltam.

A játékban minden szabadon licenszelt kivéve a hangeffektek azok non-commercial licensszel rendelkeznek.

Források

A fejlesztés során a [Godot Dokumentációt](#) számtalan alkalommal használtam. Emellett néhány esetben a [Godot Forum](#)-ból is nyertem információt főként hibákra viszont magam nem kérdeztem ott semmit.

Számos problémára kerestem jó megoldást így lehetséges hogy hasonlít egy interneten megtalálhatóéra de minden esetben van változtatás az eredetihez képest egy két scriptben bele is írtam ha valamit példának használtam (név szerint a DebugTools és az AudioAdder).

különösen ki szeretném emelni a „[Modern Exterior Ballistics - The Launch and Flight Dynamics of Symmetric Projectiles 2nd ed. - R. McCoy](#)” című könyvet mivel nagyon hasznos volt az alapok megértésében és a különböző modellek közti különbségek értelmezésében.

A Dokumentáció megírásához különösen forrást nem használtam a képek sajátok vagy például a Godot Addon GitHubján megtalálhatóak amihez hivatkozást tettem.

Irodalom Jegyzék

<https://archive.org/details/ModernExteriorBallisticsTheLaunchAndFlightDynamicsOfSymmetricProjectiles2ndEd.R.McCoy/page/n11/mode/2up> Ez különösen jó.

<https://scholar.google.com/citations?user=FbbcB6MAAAAJ> Tőle majdnem minden papírt átnéztem.

<https://repository.nwu.ac.za/bitstream/handle/10394/34069/Aldoegre%20M%2027359948.pdf?sequence=1>

<https://hrcak.srce.hr/file/379956>

https://shop.standards.ie/en-ie/standards/stanag-4355-2017-736122_saig_nato_nato_3224388/ Ennél akármelyik verzió jó. Ebben a változó nevek és mértékegységek meg mindennek a számítása benne van. És minden másik papír felsorolva ezt referenciának használja.

A legtöbb főként a 3Ds modellről szól ami a PMM és az MPMM amit végül nem sikerült implementálnom.

Utószó

Ezzel a Projecttel sokat tanultam a játékfejlesztés minden terén.

A célomat elértem ami az volt hogy csináljak realisztikusabb ballisztikát mint amit játékokban szokás használni. És mivel ezt egy Addonként hoztam létre csak az 1 mappa átmásolásával ezt akármilyen projectben lehetne használni.

És emellé egy jól kinéző 3Ds játékot is létrehoztam bemutatásnak. Lehet valamikor úgy gondolom hogy megfogok mindent amit tanultam erről és megcsinálom a második verziót. Annál viszont képes leszek csak a ballisztikán dolgozni mivel nem kell se egy komoly dokumentáció se egy játék köré. Ha a minden másra töltött időmet csak erre szántam volna talán ott lennék hogy semmit sem tudnák már belerakni maximum más fajta modelleket mint rakétákat és nem pörgés-stabilizált lövedékeket.

De akárhogy is lesz arra a problémára hogy a játékfejlesztők nem realisztikus lövedékeket használnak csináltam egy megoldást.

Van jó pár dolog ami nem került bele a végső verzióba.

Ebből talán a legnagyobb egy alternatív 3Ds ballisztikai motor ami míg jóval lassabb lenne viszont realisztikusabb is.

Szerettem volna egy több fegyvert név szerint egy sörétes puskát és valami géppuskát is a játékba. Mivel a scriptek nagy része úgy lett megírva hogy kompatibilis egy másik fegyverrel ezt egy-két nap alatt meg tudnám csinálni ebben a modellezés lenne a leghosszabb rész. Viszont mivel a mostani pálya nagy új pályát kéne csinálnom.

Söréteshez kötve míg vannak agyaggalambok nincs rendszer ami felrepíti őket a levegőbe de puskával az álló célpontokat is elég nehéz eltalálni.

Emellett szimpla papír célpontok is hiányoznak de azok nem adnának sokat a játékhoz maximum egy pontra számolt módot tudnák belőle csinálni ezt is egy nap alatt meg tudnám csinálni végül a reaktív fém célpontok érdekesebbek.

Egy másik dolog az hogy a játékosodnak nincs modelje ezért csak egy lebegő fegyver viszont humanoid modellezése és animálása kifejezetten nehéz és egy szimpla kapszulát nem akartam.

Ezen kívül főként kisebb dolgok vannak.

De van egy mondas amit nagyon szeretek. „Nem akkor tökéletes valami ha már nincs mit hozzáadni. Hanem ha már nincs mit elvenni.” nem tudom már kitől van de néha rágondolok.

Ábrajegyzék

1. ábra Godot Project Manager.....	6
2. ábra Godot Editorja 3Ds nézetben.....	6
3. ábra Godot Editorja 2Ds nézetben.....	7
4. ábra Godot beépített szövegszerkesztője.....	7
5. ábra Godot konzol ablaka.....	8
6. ábra Godot Motor fájlrendszere.....	8
7. ábra Egy Godot Scene kompozíciója	9
8. ábra Egy C#-ban írt Resource.....	10
9. ábra Az a Resource a Godot Editorban.....	10
10. ábra Terrain3D palettája	12
11. ábra Terrain felemelése.....	13
12. ábra Terrain textúrájának változtatása	13
13. ábra A teljes világ	14
14. ábra A puska modellje Blenderben	16
15. ábra Godot Consoleban lévő hibaüzenet	17
16. ábra Z-Fighting.....	17
17. ábra A menü.....	19
18. ábra Az Autoloadok listája	21
19. ábra egy Slider.....	22
20. ábra A Slider beállításai.....	23
21. ábra Egy target beállításai	23

22. ábra A puska beállításai	24
23. ábra NCBS fájlrendszere	25
24. ábra NCBS_world beállításai	25
25. ábra Golyó hozzáadása	26
26. ábra Golyó hozzáadás kódrészlete C#-ban.....	26
27. ábra Golyó inicializálása	26
28. ábra Golyó inicializálásának kódrészlete	27
29. ábra Golyó jelenlegi pozíciójának a kiválasztása	27
30. ábra Kódrészlet ami eldönti hogy a Golyó eltalált e valamit	28
31. ábra Ha nem talált el semmit a golyó forgatása és mozgatása	28
32. ábra Ha a Golyó kiesik a pályából kitöröljük.	29
33. ábra A golyó ereje ≈ 50 méteren.....	29
34. ábra 7.62x51 NATO sebessége és energiája	29
35. ábra A golyó ereje ≈ 400 méteren.....	29